

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool: Debugging & Optimisation Task (Low)

Assessment Tool Weightage: 10%

Q.No	Question	Bloom's Level
1	Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.	BL2 – Understand
2	Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue. Bloom's Level:	BL2 – Understand
3	Correct a faulty SELECT query where column names are misspelled or improperly referenced.	BL1 – Remember
4	Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.	BL3 – Apply
5	Identify and correct a subquery error where multiple values are returned instead of a single value.	BL2 – Understand
6	Debug a VIEW creation statement that contains invalid column references or syntax errors.	BL1 – Remember
7	Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.	BL3 – Apply
8	Identify violations of integrity constraints in a given table and correct them.	BL2 – Understand
9	Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies.	BL3 – Apply
10	Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.	BL3 – Apply

Code of Conduct:

I, **A Mohammad Naheed -192525336** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A Mohammad Naheed

1. Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.

Original (As per Question):

```
mysql> CREATE TABLE students
->     student_id INT,
->     name VARCHAR,
->     age INT,
->     grade,
->     PRIMARY KEY student_id
-> );
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'student_id INT,
name VARCHAR,
age INT,
grade,
PRIMARY KEY studen' at line 2
mysql> |
```

Corrected:

```
mysql> use sys;
Database changed
mysql> CREATE TABLE students (
->     student_id INT PRIMARY KEY,
->     name VARCHAR(100) NOT NULL,
->     age INT,
->     grade CHAR(2)
-> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> INSERT INTO students VALUES (1, 'Mohammad Naheed', 20, 'A');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO students VALUES (2, 'ziyaa', 21, 'B');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO students VALUES (3, 'Rohan Patel', 22, 'A');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO students VALUES (4, 'Meera Nair', 19, 'C');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> SELECT * FROM students;
+-----+-----+-----+-----+
| student_id | name          | age | grade |
+-----+-----+-----+-----+
| 1          | Mohammad Naheed | 20  | A     |
| 2          | ziyaa          | 21  | B     |
| 3          | Rohan Patel    | 22  | A     |
| 4          | Meera Nair     | 19  | C     |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Errors Fixed:

1. Missing parentheses around column definitions
2. VARCHAR missing size parameter
3. 'grade' column missing data type
4. PRIMARY KEY syntax incorrect (should be after column definition or with parentheses)

2. Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue.

Original (As per Question):

```
mysql> SELECT department, AVG(salary) FROM employees;
ERROR 1140 (42000): In aggregated query without GROUP BY, expression #1 of S
ELECT list contains nonaggregated column 'a3.employees.department'; this is
incompatible with sql_mode=only_full_group_by
mysql> |
```

Corrected:

```
mysql> SELECT department, AVG(salary) AS avg_salary
-> FROM employees
-> GROUP BY department;
+-----+-----+
| department | avg_salary |
+-----+-----+
| HR         | 37500.000000 |
| IT         | 57500.000000 |
| Sales      | 45000.000000 |
+-----+-----+
3 rows in set (0.06 sec)
```

Error Fixed: Added GROUP BY department to aggregate salaries by department

3. Correct a faulty SELECT query where column names are misspelled or improperly referenced.

Original (As per Question):

```
mysql> SELECT emp_name, depatment, sallary FROM employees;
ERROR 1054 (42S22): Unknown column 'emp_name' in 'field list'
mysql> |
```

Corrected:

```
mysql> SELECT emp_name, department, salary
-> FROM employees;
+-----+-----+-----+
| emp_name | department | salary |
+-----+-----+-----+
| Rahul    | HR         | 35000.00 |
| Ayesha   | HR         | 40000.00 |
| Kiran    | IT         | 55000.00 |
| Sneha    | IT         | 60000.00 |
| Arjun    | Sales      | 45000.00 |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

Error Fixed: Corrected column names: emp_name→name, depatment→department, sallary→salary

4. Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.

Original (As per Question):

```
mysql> SELECT students.name, courses.course_name
      -> FROM students
      -> JOIN enrollments;
ERROR 1054 (42S22): Unknown column 'courses.course_name' in 'field list'
mysql> |
```

Corrected:

```
mysql> SELECT students.name, courses.course_name
      -> FROM students
      -> JOIN enrollments
      -> ON students.student_id = enrollments.student_id
      -> JOIN courses
      -> ON enrollments.course_id = courses.course_id;
+-----+-----+
| name          | course_name |
+-----+-----+
| Mohammad Naheed | Math        |
| Mohammad Naheed | Science     |
| Ayesha         | English     |
| Rohan Patel    | Math        |
+-----+-----+
4 rows in set (0.00 sec)
```

Error Fixed: Added proper JOIN conditions

5. Identify and correct a subquery error where multiple values are returned instead of a single value

Original (As per Question):

```
mysql> SELECT name FROM studentss
      -> WHERE student_id = (SELECT student_id FROM enrollments WHERE course_id = 101);
ERROR 1242 (21000): Subquery returns more than 1 row
mysql> |
```

Corrected:

```
mysql> SELECT name FROM students
      -> WHERE student_id IN (SELECT student_id FROM enrollments WHERE course_id = 101);
+-----+
| name          |
+-----+
| Mohammad Naheed |
| Rohan Patel    |
+-----+
2 rows in set (0.02 sec)
```

Error Fixed: Changed '=' to 'IN' when subquery returns multiple values

6. Debug a VIEW creation statement that contains invalid column references or syntax errors.

Original (As per Question):

```
mysql> CREATE VIEW student_courses AS
-> SELECT s.name, c.course_name
-> FROM students s
-> JOIN enrollments e ON s.student_id = e.id
-> JOIN courses c ON e.course_id = c.course_code;
ERROR 1054 (42S22): Unknown column 'c.course_code' in 'on clause'
mysql> |
```

Corrected:

```
mysql> CREATE VIEW student_courses AS
-> SELECT s.name, c.course_name
-> FROM students s
-> JOIN enrollments e ON s.student_id = e.student_id
-> JOIN courses c ON e.course_id = c.course_id;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM student_courses;
+-----+-----+
| name          | course_name |
+-----+-----+
| Mohammad Naheed | Math        |
| Mohammad Naheed | Science     |
| Ayesha         | English     |
| Rohan Patel    | Math        |
+-----+-----+
4 rows in set (0.00 sec)
```

Errors Fixed:

1. Corrected e.id to e.student_id
2. Corrected c.course_code to c.course_id

7. Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.

Original (As per Question):

```
mysql> SELECT * FROM products
-> WHERE price > 100
-> AND price >= 50
-> AND category = 'Electronics'
-> OR category = 'Electronics';
```

product_id	name	category	price
1	Laptop	Electronics	1200.00
2	Headphones	Electronics	80.00
4	Monitor	Electronics	350.00

3 rows in set (0.00 sec)

Corrected:

```
mysql> SELECT * FROM products
-> WHERE price > 100 AND category = 'Electronics';
```

product_id	name	category	price
1	Laptop	Electronics	1200.00
4	Monitor	Electronics	350.00

2 rows in set (0.00 sec)

Optimisation: Removed redundant conditions (price >= 50 was redundant with price > 100, and duplicate category condition)

8. Identify violations of integrity constraints in a given table and correct them.

Original (As per Question):

```
mysql> CREATE TABLE orders (  
->     order_id INT PRIMARY KEY,  
->     customer_id INT NOT NULL,  
->     order_date DATE,  
->     status VARCHAR(20) CHECK (status IN ('Pending', 'Shipped', 'Deliv  
ered'))  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO orders VALUES (1, NULL, '2024-01-01', 'Processing');  
ERROR 1048 (23000): Column 'customer_id' cannot be null  
mysql> INSERT INTO orders VALUES (1, 101, '2024-01-02', 'Shipped');  
Query OK, 1 row affected (0.01 sec)
```

Corrected:

```
mysql> INSERT INTO orders VALUES (1, NULL, '2024-01-01', 'Processing');  
ERROR 1048 (23000): Column 'customer_id' cannot be null  
mysql> INSERT INTO orders VALUES (1, 101, '2024-01-02', 'Shipped');  
Query OK, 1 row affected (0.01 sec)  
  
mysql> INSERT INTO orders VALUES (1, 101, '2024-01-01', 'Pending');  
ERROR 1062 (23000): Duplicate entry '1' for key 'orders.PRIMARY'  
mysql> INSERT INTO orders VALUES (2, 102, '2024-01-02', 'Shipped');  
Query OK, 1 row affected (0.01 sec)  
  
mysql> INSERT INTO orders VALUES (3, 103, '2024-01-03', 'Delivered');  
Query OK, 1 row affected (0.01 sec)
```

Errors Fixed:

1. customer_id cannot be NULL (NOT NULL constraint)
2. status must be one of: 'Pending', 'Shipped', 'Delivered' (CHECK constraint)
3. order_id must be unique (PRIMARY KEY constraint)

9. Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies.

Original (As per Question):

```
mysql> CREATE TABLE employee_projects (  
->     emp_id INT,  
->     emp_name VARCHAR(100),  
->     project1 VARCHAR(50),  
->     project2 VARCHAR(50),  
->     project3 VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

Corrected:

```
mysql> CREATE TABLE employees (  
->     emp_id INT PRIMARY KEY,  
->     emp_name VARCHAR(100)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> CREATE TABLE projects (  
->     project_id INT PRIMARY KEY,  
->     project_name VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.02 sec)  
  
mysql> CREATE TABLE employee_project_assignments (  
->     emp_id INT,  
->     project_id INT,  
->     assignment_date DATE,  
->     PRIMARY KEY (emp_id, project_id),  
->     FOREIGN KEY (emp_id) REFERENCES employees(emp_id),  
->     FOREIGN KEY (project_id) REFERENCES projects(project_id)  
-> );  
Query OK, 0 rows affected (0.07 sec)  
  
mysql> INSERT INTO employees VALUES (1, 'John Doe'), (2, 'Jane Smith');  
Query OK, 2 rows affected (0.01 sec)  
Records: 2 Duplicates: 0 Warnings: 0  
  
mysql> INSERT INTO projects VALUES (101, 'Alpha'), (102, 'Beta'), (103, 'Gamma');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
mysql> INSERT INTO employee_project_assignments VALUES (1, 101, '2024-01-01');  
Query OK, 1 row affected (0.01 sec)  
  
mysql> INSERT INTO employee_project_assignments VALUES (1, 102, '2024-01-15');  
Query OK, 1 row affected (0.00 sec)  
  
mysql> INSERT INTO employee_project_assignments VALUES (2, 103, '2024-02-01');  
Query OK, 1 row affected (0.01 sec)
```

Rectification:

- 1NF: Eliminated repeating groups (project1, project2, project3)
- 2NF: Created separate tables to remove partial dependencies

10. Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.

Original (As per Question):

```
mysql> CREATE TABLE order_details (
->   order_id INT,
->   product_id INT,
->   product_name VARCHAR(100),
->   product_price DECIMAL(10,2),
->   quantity INT,
->   customer_id INT,
->   customer_name VARCHAR(100),
->   customer_city VARCHAR(50)
-> );
Query OK, 0 rows affected (0.03 sec)
```

Corrected:

```
mysql> CREATE TABLE customers (
->   customer_id INT PRIMARY KEY,
->   customer_name VARCHAR(100),
->   customer_city VARCHAR(50)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE products (
->   product_id INT PRIMARY KEY,
->   product_name VARCHAR(100),
->   product_price DECIMAL(10,2)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE orders (
->   order_id INT PRIMARY KEY,
->   customer_id INT,
->   order_date DATE,
->   FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
-> );
Query OK, 0 rows affected (0.10 sec)

mysql> CREATE TABLE order_items (
->   order_id INT,
->   product_id INT,
->   quantity INT,
->   PRIMARY KEY (order_id, product_id),
->   FOREIGN KEY (order_id) REFERENCES orders(order_id),
->   FOREIGN KEY (product_id) REFERENCES products(product_id)
-> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO customers VALUES (1, 'Alice Johnson', 'New York');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO customers VALUES (2, 'Bob Wilson', 'Los Angeles');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> INSERT INTO products VALUES (101, 'Laptop', 1200.00);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO products VALUES (102, 'Mouse', 25.00);
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO orders VALUES (1001, 1, '2024-01-01');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO orders VALUES (1002, 2, '2024-01-02');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> INSERT INTO order_items VALUES (1001, 101, 1);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO order_items VALUES (1001, 102, 2);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO order_items VALUES (1002, 101, 1);
Query OK, 1 row affected (0.00 sec)
```

Rectification:

- Eliminated redundant customer and product data
- Separated into logical tables (Customers, Products, Orders, OrderItems)
- product_price depends on product_id, not order_id

